Revision Games — 2.1 Elements of Computational Thinking

OCR A Level Computer Science (H446) — Component 02 — Spec ref 2.1.1–2.1.5

> This topic is examined by **application to a scenario**. These games drill the vocabulary and the apply-it reasoning behind the five thinking skills.

Loop cards (dominoes)

How to play: Print and cut out the 16 cards. Deal them out among players. One player reads the **ANSWER** on the bottom of any card. Whoever holds the card whose **TOP (question/clue)** matches that answer reads it out, then reads their own bottom answer. Play continues until the chain returns to the start — the loop is closed and verified. (Card 16's answer leads back to Card 1's clue.)

Card	THIS card's answer (top)	… reads its clue (bottom): "Who has …"
1	**Abstraction**	… the skill of breaking a problem into an ordered sequence of smaller sub-problems?
2	**Decomposition**	… planning before coding: identifying inputs, outputs, preconditions and reusable parts?
3	**Thinking ahead**	… a condition that must be true *before* a routine can run correctly?
4	**Precondition**	… storing expensive or frequently used results in fast storage for reuse?
5	**Caching**	… cached data that is now out of date because the source has changed?
6	**Stale data**	… a self-contained, generalised module that can be used in many programs?
7	**Reusable component**	… the skill of identifying where the flow of control branches on a condition?
8	**Thinking logically**	… a place in a program where execution branches, e.g. an IF or CASE?
9	**Decision point**	… the order in which statements execute, shaped by those branches?
10	**Flow of control**	… the skill of spotting parts that can run at the same time?
11	**Thinking concurrently**	… a fault where the result depends on the unpredictable timing of shared tasks?
12	**Race condition**	… when tasks each wait for a resource the other holds, so none proceed?
13	**Deadlock**	… multiple tasks running literally simultaneously on multiple cores?
14	**Parallel processing**	… abstraction that removes detail from one specific thing to model that problem?
15	**Representational abstraction**	… abstraction that groups many things by shared characteristics into a general model?
16	**Generalisation (categorisation)**	… removing or hiding unnecessary detail so only relevant information remains?

Loop check: 16 → "removing/hiding unnecessary detail" → Card 1 (Abstraction). Closed.

Taboo cards

How to play: In pairs/teams. One player describes the **TERM** at the top of the card to their team **without saying any of the four BANNED words** (or obvious variants). One point per correct guess; if a banned word slips out, the card is discarded. ~45 seconds per card.

Card 1 — TERM: ABSTRACTION Banned: detail · remove · hide · simplify

Card 2 — TERM: DECOMPOSITION Banned: break · smaller · sub-problem · parts

Card 3 — TERM: PRECONDITION Banned: before · true · must · runs

Card 4 — TERM: CACHING Banned: store · fast · reuse · results

Card 5 — TERM: CONCURRENCY Banned: same time · parallel · together · simultaneous

Card 6 — TERM: RACE CONDITION Banned: timing · shared · order · unpredictable

Card 7 — TERM: REUSABLE COMPONENT Banned: module · again · self-contained · library

Card 8 — TERM: DECISION POINT Banned: IF · branch · condition · flow

---

'Apply it' challenge cards

How to play: Shuffle and draw a card. It names a **SCENARIO** and a **THINKING SKILL**. The player must give a **valid, applied** answer using that scenario's actual details (not a generic definition). The group checks it against the example answer; award the point for any equally valid response. This mirrors the AO2/AO3 exam style.

Card 1 — Scenario: online chess game · Skill: thinking logically Give one decision point with an exact Boolean and both branches. *Example:* `IF move is legal` → TRUE: update the board and switch player; FALSE: reject the move and ask again.

Card 2 — Scenario: weather app · Skill: caching Explain one use of caching plus a trade-off. *Example:* Cache the latest forecast so repeated opens load instantly and cut API calls; trade-off: it can go **stale** if the forecast updates, so refresh on a timer.

Card 3 — Scenario: ride-booking app (e.g. taxis) · Skill: thinking concurrently Name two tasks that can run at once and one trade-off. *Example:* Track the car's GPS **while** the UI updates the map; trade-off: harder to debug and risk of inconsistent state needing synchronisation.

Card 4 — Scenario: self-checkout till · Skill: thinking ahead (inputs/outputs) State two inputs and one output. *Example:* Inputs — scanned barcode, payment; Output — receipt / "payment accepted" message.

Card 5 — Scenario: a road satnav · Skill: thinking abstractly Name one detail kept and one removed, with reason. *Example:* Keep road connections and distances; remove the colour of the buildings — irrelevant to finding the shortest route.

Card 6 — Scenario: school timetable generator · Skill: thinking procedurally Name three ordered sub-procedures. *Example:* 1) Read teacher/room availability; 2) Assign classes to free slots; 3) Output the finished timetable.

Card 7 — Scenario: ATM / cash machine · Skill: thinking ahead (precondition) State a precondition that must hold before dispensing cash. *Example:* The account balance must be ≥ the requested amount (and the PIN already verified).

Card 8 — Scenario: photo-editing software · Skill: reusable components Give one reusable component and a benefit. *Example:* A "blur" filter module reused by many tools — faster development and fewer bugs as it is already tested.

Card 9 — Scenario: multiplayer quiz app · Skill: thinking concurrently (trade-off) Identify a risk of letting players answer simultaneously. *Example:* A **race condition** when two answers update the shared scoreboard at once — needs locking/synchronisation; not automatically faster.

Card 10 — Scenario: e-commerce checkout · Skill: thinking logically Give one decision point with an exact Boolean and both branches. *Example:* `IF stockLevel >= quantityOrdered` → TRUE: confirm the order and reduce stock; FALSE: show "out of stock".

Quick-fire 'Name the thinking skill'

How to play: Read each clue aloud; players race to name the correct **thinking skill** (or term). First correct answer wins the point. Keep it fast — about 10 seconds per clue.

#	Clue	Answer
1	"I removed the room's carpet colour from my model because it doesn't affect bookings."	Thinking abstractly
2	"I worked out the inputs, outputs and what must be true first, before writing any code."	Thinking ahead
3	"I split 'make a booking' into check, store and confirm steps in a fixed order."	Thinking procedurally
4	"I wrote `IF seatsRequested <= seatsAvailable` and planned both branches."	Thinking logically
5	"I spotted that logging and rendering the screen have no data dependency, so they can overlap."	Thinking concurrently
6	"The list must be sorted before this binary search will work."	Precondition
7	"I stored the frequently requested results in fast memory to avoid recomputing them."	Caching
8	"My cached price list is wrong because the database changed but the cache didn't."	Stale data
9	"Two students booked the same slot at the same instant and the result depended on timing."	Race condition
10	"I grouped savings and current accounts into one general `Account` model."	Generalisation / categorisation (abstraction)
11	"Each task is waiting for a resource the other is holding, so nothing moves."	Deadlock
12	"I made one tested, self-contained module and used it across three programs."	Reusable component

---

Accuracy note: definitions and trade-offs follow the OCR H446 spec (2.1.1 abstraction & generalisation; 2.1.2 inputs/outputs, preconditions, caching, reusable components; 2.1.3 procedural decomposition; 2.1.4 decision points/flow of control; 2.1.5 concurrency benefits and trade-offs incl. race conditions and deadlock).